Aim - 2. Data Ingestion and Cleaning: Acquire data and prepare it for analysis.

Theory

1. What is data ingestion and why is it important in data science? Which Python library is commonly used for data ingestion and cleaning?

Data ingestion is the process of collecting and importing data from various sources such as databases, CSV files, APIs, or web services into a system for further processing and analysis. It is a fundamental step in the data science pipeline because the quality and availability of data directly impact the accuracy and effectiveness of analysis and machine learning models. Without proper data ingestion, it is not possible to work with real-world data efficiently. In Python, one of the most commonly used libraries for data ingestion and cleaning is Pandas. It provides powerful data structures like DataFrames and functions that make it easy to read, manipulate, and clean data from different formats such as CSV, Excel, and JSON files.

2. Why is data cleaning necessary before performing analysis or machine learning?

Data cleaning is necessary because real-world data is often incomplete, inconsistent, and contains errors such as missing values, duplicate records, or incorrect formats. If such unclean data is used directly for analysis or machine learning, it can lead to inaccurate results, poor model performance, and misleading conclusions. Data cleaning ensures that the dataset is reliable, consistent, and properly formatted, which improves the quality of insights and predictions. It involves tasks like handling missing values, removing duplicates, correcting errors, and standardizing data formats. Therefore, data cleaning is a critical step to ensure the success of any data science project.

3. How do you load a CSV file into a Pandas DataFrame?

To load a CSV file into a Pandas DataFrame, the read_csv() function is used. This function reads the data from a CSV file and converts it into a structured DataFrame, which allows easy manipulation and analysis. The user needs to first import the Pandas library and then specify the file path of the CSV file. For example, using pd.read_csv("file_name.csv") loads the file into a DataFrame. This method is widely used because it is simple, efficient, and supports various parameters such as handling delimiters, headers, and missing values during the loading process.

4. What is the difference between head() and tail() functions in Pandas?

In Pandas, the head() and tail() functions are used to quickly view a subset of data from a DataFrame. The head() function returns the first few rows of the dataset, typically the first five rows by default, which helps in understanding the structure and initial data entries. On the other hand, the tail() function returns the last few rows of the dataset, also five rows by default, which is useful for inspecting the end of the data. Both functions are helpful for quickly checking data without displaying the entire dataset, especially when working with large datasets.

5. How can you identify missing values in a dataset?

Missing values in a dataset can be identified using various functions provided by the Pandas library. The most commonly used method is the isnull() or isna() function, which returns a boolean DataFrame indicating the presence of missing values. To count the total number of missing values in each column, the sum() function can be used along with isnull(), such as df.isnull().sum(). Additionally, functions like info() can provide a summary of non-null values in each column, helping to detect missing data. Identifying missing values is an important step in data cleaning, as it allows the user to decide how to handle them, such as removing or imputing the missing entries.

```
import pandas as pd
import numpy as np
```

Loaded a CSV file using pandas and display first five rows.

```
from google.colab import files
uploaded = files.upload()
```

Choose files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving titanic.csv to titanic.csv
```

```
df = pd.read_csv(list(uploaded.keys())[0])
```

```
df.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	T
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC
2	3	1	3	Heikkinen, Miss.	female	26.0	0	0	ST(3
3	4	1	1	Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	3

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    object
 4   Sex          891 non-null    object
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    object
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object
 11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

Identifying the missing value in dataset

```
df.isnull().sum()
```

	0
PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	177
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	687
Embarked	2

dtype: int64

3. Replaceing the missing values with zero

```
df = df.fillna(0)
```

```
df.isnull().sum()
```

	0
PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	0
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	0
Embarked	0

dtype: int64

4. Rename column names to lower case.

```
df.columns = df.columns.str.lower()
df.head()
```

	passengerid	survived	pclass	name	sex	age	sibsp	parch	t
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	ST(3
				Futrelle, Mrs. (Lily May Peel)					
4	5	0	3	Allen, Mr. William	male	35.0	0	0	3

5. Display Dataset shape (rows and columns)

```
print("Rows and Columns:", df.shape)
```

```
Rows and Columns: (891, 12)
```

6. Remove Duplicate rows from a dataset

```
df = df.drop_duplicates()
df.shape
```

```
(891, 12)
```

7. Handel missing values using mean or median

```
df.fillna(df.median(numeric_only=True), inplace=True)
```

8. Change Data type of Selected column

```
df['age'] = df['age'].astype(int)
df.dtypes
```

	0
passengerid	int64
survived	int64
pclass	int64
name	object
sex	object
age	int64
sibsp	int64
parch	int64
ticket	object
fare	float64
cabin	object
embarked	object

dtype: object

9. Filter rows based on a given condition

```
filtered_df = df[df['age'] > 30]
filtered_df.head()
```

	passengerid	survived	pclass	name	sex	age	sibsp	parch	ti
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38	1	0	1
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35	1	0	11
11	12	1	1	Bonnell, Miss. Elizabeth	female	58	0	0	11

10. Save a cleaned dataset to a new CSV file

```
df.to_csv("cleaned_dataset.csv", index=False)
```

```
from google.colab import files
files.download("cleaned_dataset.csv")
```

11. Compare Missing Value Meethod

Task

Compare different missing value imputation strategies, including 0-imputation and median-imputation, analyze their impact on data distributions, clean inconsistent categorical values (e.g., 'Sex', 'Embarked'), detect and handle outliers in numerical columns (e.g., 'Age', 'Fare'), encapsulate these data cleaning steps into reusable functions, and generate a comprehensive report summarizing all data cleaning actions.

Compare Missing Value Handling Strategies

Subtask:

Apply and compare different missing value handling strategies (0-imputation and median-imputation) on the dataset, and analyze their impact on the data distribution.

Reasoning: The first step is to reload the original `titanic.csv` dataset into a new DataFrame named `df_original` to ensure we start with the original missing

values, and then convert its column names to lowercase as specified in the instructions.

```
df_original = pd.read_csv(list(uploaded.keys())[0])
df_original.columns = df_original.columns.str.lower()
df_original.head()
```

	passengerid	survived	pclass	name	sex	age	sibsp	parch	t
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	ST(3
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	
				Allen, Mr.					

Reasoning: The previous code block failed because 'pd' was not defined. This indicates that the pandas library, which was imported in a prior cell, is not available in the current execution environment. I need to re-import pandas to resolve this `NameError` and then proceed with reloading the data and converting column names to lowercase.

```
import pandas as pd

df_original = pd.read_csv(list(uploaded.keys())[0])
df_original.columns = df_original.columns.str.lower()
df_original.head()
```

	passengerid	survived	pclass	name	sex	age	sibsp	parch	t
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	ST(3
3	4	1	1	Futrelle, Mrs. Jacques Heath	female	35.0	1	0	

Reasoning: The 'uploaded' variable from `files.upload()` is not defined across kernel restarts or different execution contexts. To fix this, I need to directly specify

the filename 'titanic.csv', assuming it was uploaded and saved in the current working directory.

```
import pandas as pd

df_original = pd.read_csv('titanic.csv')
df_original.columns = df_original.columns.str.lower()
df_original.head()
```

passengerid	survived	pclass	name	sex	age	sibsp	parch	t